Anomaly Detection Using an Autoencoder

Dataset: MNIST handwritten digits (used here to demonstrate reconstruction-based anomaly detection)
Framework: TensorFlow and Keras

Aim

Implement an autoencoder for anomaly detection using these stages: a. Import required libraries
b. Load/access the dataset
c. Encode the input into a latent representation
d. Decode the latent representation to reconstruct the input
e. Compile, train, and evaluate the model using an optimizer, loss function, and evaluation metrics

Approach: Train the autoencoder using only images of digit **0** as normal data. During testing, images of other digits are treated as anomalies. The reconstruction error is used as the anomaly score.

a. Import required libraries

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras import Model, layers
from tensorflow.keras.datasets import mnist

print("TensorFlow version:", tf.__version__)
```

```
TensorFlow version: 2.21.0
```

b. Load and preprocess the dataset

```
# Load the MNIST dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normalize pixel values to the range 0 to 1
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Flatten each 28 x 28 image into a vector of 784 features
x_train = x_train.reshape((-1, 784))
x_test = x_test.reshape((-1, 784))

# Use only digit 0 images to train the autoencoder (normal samples)
x_train_normal = x_train[y_train == 0]

print("Normal training samples:", x_train_normal.shape)
print("Test samples:", x_test.shape)

# Define test labels: 0 = normal digit 0, 1 = anomaly (other digits)
y_test_anomaly = (y_test != 0).astype("int32")
print("Normal test samples:", np.sum(y_test_anomaly == 0))
print("Anomalous test samples:", np.sum(y_test_anomaly == 1))
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Normal training samples: (5923, 784)
Test samples: (10000, 784)
Normal test samples: 980
Anomalous test samples: 9020
```

c. Encoder: convert input into a latent representation

```
input_dim = 784
latent_dim = 32

# Encoder maps 784 input values into a 32-dimensional latent vector
encoder_input = layers.Input(shape=(input_dim,), name="encoder_input")
x = layers.Dense(128, activation="relu")(encoder_input)
x = layers.Dense(64, activation="relu")(x)
latent_vector = layers.Dense(latent_dim, activation="relu", name="latent_vector")(x)
```

```
encoder = Model(encoder_input, latent_vector, name="encoder")
encoder.summary()
```

```
Model: "encoder"
```

Layer (type)	Output Shape	Param #
encoder_input (InputLayer)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
latent_vector (Dense)	(None, 32)	2,080

```
Total params: 110,816 (432.88 KB)
Trainable params: 110,816 (432.88 KB)
Non-trainable params: 0 (0.00 B)
```

d. Decoder: reconstruct the original input

```
# Decoder maps the 32-dimensional latent vector back to 784 values
decoder_input = layers.Input(shape=(latent_dim,), name="decoder_input")
x = layers.Dense(64, activation="relu")(decoder_input)
x = layers.Dense(128, activation="relu")(x)
reconstructed_output = layers.Dense(input_dim, activation="sigmoid")(x)

decoder = Model(decoder_input, reconstructed_output, name="decoder")
decoder.summary()

# Combine encoder and decoder into one autoencoder
autoencoder_input = layers.Input(shape=(input_dim,), name="autoencoder_input")
encoded = encoder(autoencoder_input)
decoded = decoder(encoded)
autoencoder = Model(autoencoder_input, decoded, name="autoencoder")

autoencoder.summary()
```

```
Model: "decoder"
```

Layer (type)	Output Shape	Param #
decoder_input (InputLayer)	(None, 32)	0
dense_2 (Dense)	(None, 64)	2,112
dense_3 (Dense)	(None, 128)	8,320
dense_4 (Dense)	(None, 784)	101,136

```
Total params: 111,568 (435.81 KB)
Trainable params: 111,568 (435.81 KB)
Non-trainable params: 0 (0.00 B)
Model: "autoencoder"
```

Layer (type)	Output Shape	Param #
autoencoder_input (InputLayer)	(None, 784)	0
encoder (Functional)	(None, 32)	110,816
decoder (Functional)	(None, 784)	111,568

```
Total params: 222,384 (868.69 KB)
Trainable params: 222,384 (868.69 KB)
Non-trainable params: 0 (0.00 B)
```

e. Compile and train the autoencoder

```
# Optimizer: Adam; Loss: mean squared error (MSE)
# Metric: mean absolute error (MAE)
autoencoder.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss="mse",
    metrics=["mae"]
)

history = autoencoder.fit(
    x_train_normal,
    x_train_normal,          # Target is the same input image
    epochs=10,
    batch_size=128,
```

```
    validation_split=0.1,
    shuffle=True,
    verbose=1
)
```

```
Epoch 1/10
42/42 ━━━━━━━━━━━━━━━━━━━━ 3s 16ms/step - loss: 0.1042 - mae: 0.2201 - val_loss: 0.0625 - val_mae: 0.1332
Epoch 2/10
42/42 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - loss: 0.0550 - mae: 0.1228 - val_loss: 0.0441 - val_mae: 0.1042
Epoch 3/10
42/42 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0416 - mae: 0.0992 - val_loss: 0.0375 - val_mae: 0.0918
Epoch 4/10
42/42 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - loss: 0.0356 - mae: 0.0892 - val_loss: 0.0318 - val_mae: 0.0815
Epoch 5/10
42/42 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0304 - mae: 0.0789 - val_loss: 0.0277 - val_mae: 0.0731
Epoch 6/10
42/42 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 0.0270 - mae: 0.0719 - val_loss: 0.0252 - val_mae: 0.0683
Epoch 7/10
42/42 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - loss: 0.0249 - mae: 0.0678 - val_loss: 0.0236 - val_mae: 0.0653
Epoch 8/10
42/42 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - loss: 0.0232 - mae: 0.0644 - val_loss: 0.0218 - val_mae: 0.0614
Epoch 9/10
42/42 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - loss: 0.0213 - mae: 0.0609 - val_loss: 0.0200 - val_mae: 0.0585
Epoch 10/10
42/42 ━━━━━━━━━━━━━━━━━━━━ 1s 16ms/step - loss: 0.0193 - mae: 0.0568 - val_loss: 0.0183 - val_mae: 0.0543
```

Evaluate reconstruction and detect anomalies

```
# Evaluate reconstruction quality on the complete test set
test_loss, test_mae = autoencoder.evaluate(x_test, x_test, verbose=0)
print(f"Test reconstruction loss (MSE): {test_loss:.6f}")
print(f"Test reconstruction MAE: {test_mae:.6f}")

# Reconstruction error for each test image (mean squared error per image)
x_test_reconstructed = autoencoder.predict(x_test, verbose=0)
reconstruction_error = np.mean(np.square(x_test - x_test_reconstructed), axis=1)

# Select a threshold using the 95th percentile of training-normal errors
normal_reconstructed = autoencoder.predict(x_train_normal, verbose=0)
normal_error = np.mean(np.square(x_train_normal - normal_reconstructed), axis=1)
threshold = np.percentile(normal_error, 95)

# Error above threshold => anomaly
y_pred_anomaly = (reconstruction_error > threshold).astype("int32")

print(f"Anomaly threshold: {threshold:.6f}")
print(f"Detected anomalies: {np.sum(y_pred_anomaly == 1)} out of {len(y_pred_anomaly)} test images")

# Basic metrics
true_positive = np.sum((y_pred_anomaly == 1) & (y_test_anomaly == 1))
true_negative = np.sum((y_pred_anomaly == 0) & (y_test_anomaly == 0))
false_positive = np.sum((y_pred_anomaly == 1) & (y_test_anomaly == 0))
false_negative = np.sum((y_pred_anomaly == 0) & (y_test_anomaly == 1))

accuracy = (true_positive + true_negative) / len(y_test_anomaly)
precision = true_positive / (true_positive + false_positive + 1e-8)
recall = true_positive / (true_positive + false_negative + 1e-8)

print(f"Detection accuracy: {accuracy * 100:.2f}%")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
```

```
Test reconstruction loss (MSE): 0.057787
Test reconstruction MAE: 0.129067
Anomaly threshold: 0.033442
Detected anomalies: 8952 out of 10000 test images
Detection accuracy: 98.22%
Precision: 0.9939
Recall: 0.9864
```

Visualize training and reconstruction results

```
# Plot training and validation loss
plt.figure(figsize=(8, 5))
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.title("Autoencoder Training and Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Mean Squared Error")
plt.legend()
plt.grid(True)
```

```
plt.grid(True)
plt.tight_layout()
plt.show()

# Display examples of normal and anomalous test samples
sample_indices = []
for wanted_label in [0, 1]:
    candidates = np.where(y_test_anomaly == wanted_label)[0]
    if len(candidates):
        sample_indices.extend(candidates[:5].tolist())

plt.figure(figsize=(12, 5))
for position, idx in enumerate(sample_indices):
    # Original image
    plt.subplot(2, 10, position + 1)
    plt.imshow(x_test[idx].reshape(28, 28), cmap="gray")
    plt.title("Original\n" + ("Anomaly" if y_test_anomaly[idx] else "Normal"), fontsize=8)
    plt.axis("off")

    # Reconstructed image
    plt.subplot(2, 10, position + 11)
    plt.imshow(x_test_reconstructed[idx].reshape(28, 28), cmap="gray")
    detected = "Anomaly" if y_pred_anomaly[idx] else "Normal"
    plt.title(f"Recon.\n{detected}", fontsize=8)
    plt.axis("off")

plt.tight_layout()
plt.show()
```

